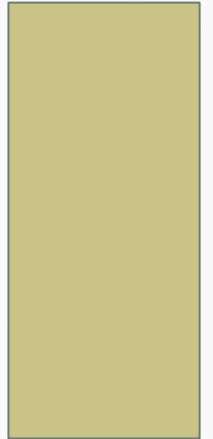


DATA STRUCTURES

LECTURE 04



BOOK READINGS

- Data Structures and Algorithms in C++
by Michael T. Goodrich, Roberto Tamassia, David M.
Mount Second Edition : Chapter 2 (2.3)

A GLIMPSE FROM PREVIOUS LECTURE

- StaticIntPtr Task
 - Error Solution (DynamicIntPtr Task)
 - Avoid Memory Leakage
- Destructor
- What are dangling pointers?

CONSTRUCTORS

- **ClassName obj1;** // Default Constructor
- **ClassName obj2(Parameter1, Parameter2,, Parameter N) ;** // Parameterized constructor
- **ClassName obj3 = obj2;** // Copy constructor

COPY CONSTRUCTOR

Copy Constructor is a type of constructor which is used to create a copy of an already existing object of a class type.

EXAMPLE

Class circle	void main()
<pre>{ float rad; public: circle() { rad=0;} circle(float r){rad=r} ~circle() {} };</pre>	<pre>{ circle obj; // obj.circle() circle obj2(6.6); // obj2.circle(6.6) circle obj3=obj2; // obj2.circle(obj2) }</pre>

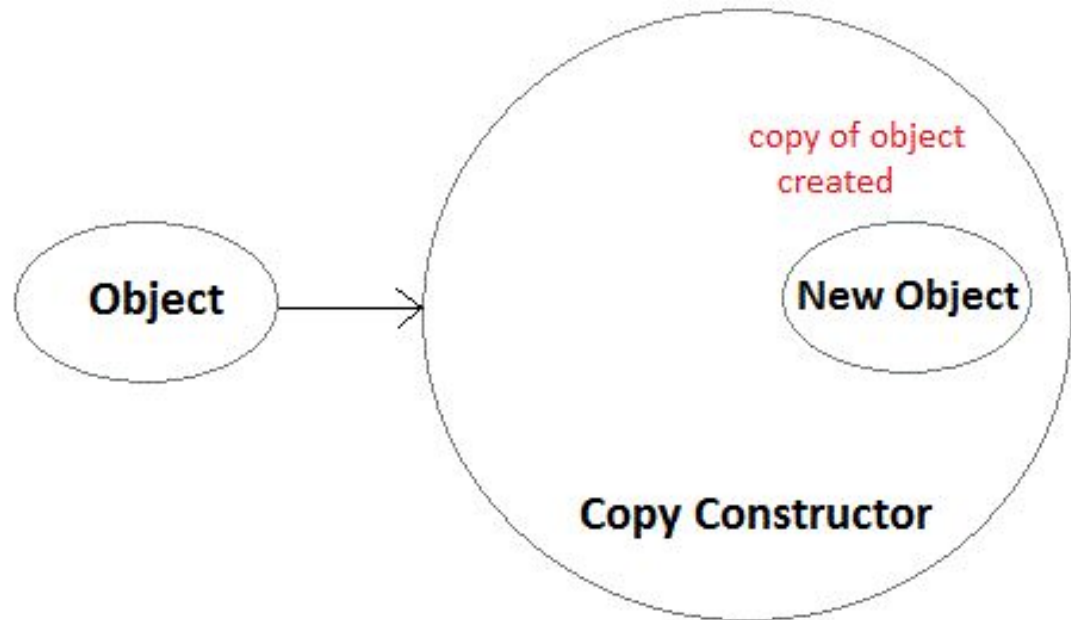
SYNTAX OF COPY CONSTRUCTOR

Classname(**const** classname & objectname)

{

....

}



EXAMPLE

Class circle	void main()
<pre>{ float rad; public: circle() { rad=0;} circle(float r){rad=r} circle (const circle & obj) { rad=obj.rad; } ~circle() {} };</pre>	<pre>{ circle obj; // obj.circle() circle obj2(6.6); // obj2.circle(6.6) circle obj3=obj2; // obj2.circle(obj2) }</pre>

COPYING POINTERS

```
class dataType
{
    int * ptr;

public:
    dataType() { ptr=NULL;}
    void set (int val)
    {
        if(ptr==NULL)
            ptr=new int;
        *ptr=val;
    }
    ~circle()
    { delete ptr; ptr=NULL; }

    Int get()
    { return *ptr;}
```

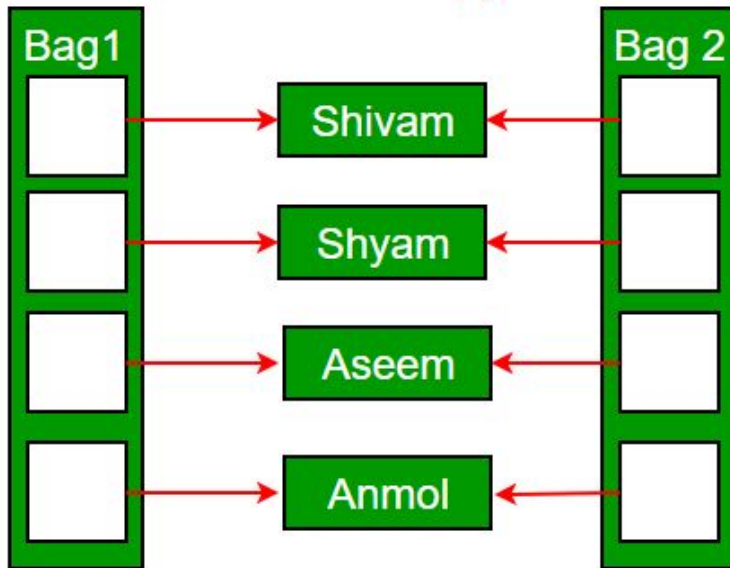
```
void main()
{

    dataType obj;
    Obj.set(5);
    datatype obj2=obj;
    cout<<obj2.get();

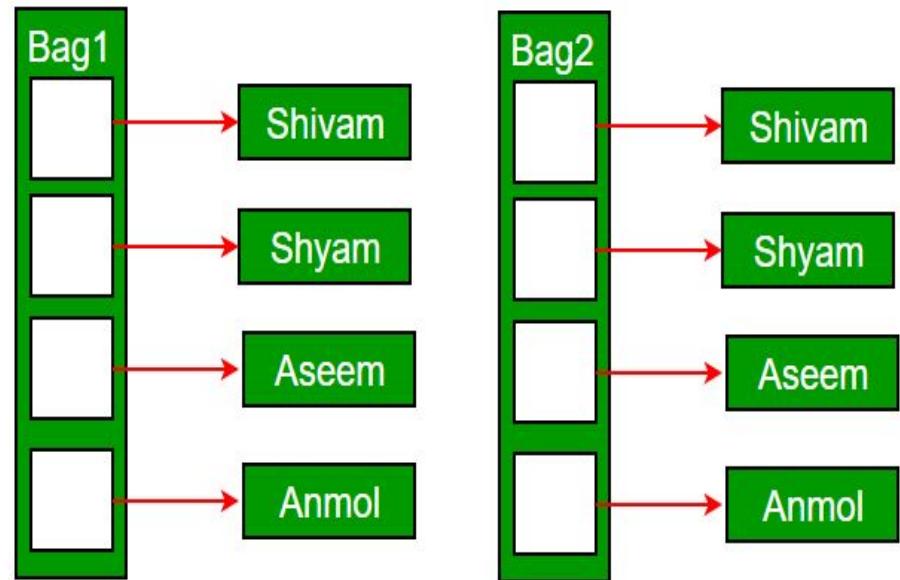
}
```

DEEP VS. SHALLOW

Shallow Copy



Deep Copy



DEEP COPY VS SHALLOW COPY

Shallow Copy

```
Void main()  
{  
int *p,*q;  
p=new int;  
*p=5;  
q=p; // shallow copy  
*q=9;  
cout<<*p;  
}
```

Deep Copy

```
Void main()  
{  
int *p,*q;  
p=new int;  
*p=5;  
q=new int;  
*q=*p; // deep copy  
cout<<*p;  
}
```

WRITE COPY CONSTRUCTOR?

```
class dataType
{
    int * ptr;

public:
    dataType() { ptr=NULL;}
    void set (int val)
    {
        if(ptr==NULL)
            ptr=new int;
        *ptr=val;
    }
    ~circle()
    { delete ptr; ptr=NULL; }

    Int get()
    { return *ptr;}
```

```
void main()
{

    dataType obj;
    Obj.set(5);
    datatype obj2=obj;
    cout<<obj2.get();

}
```

COPY CONSTRUCTOR CALL

- Called whenever
 - an object is created by initializing it with already existing object of same class
 - An object is passed as an argument to a function
 - An object is returned from a function

EXAMPLE

```
class dataType
{
    int * ptr;

public:
    dataType() { ptr=NULL;}
    void set (int val)
    {
        if(ptr==NULL)
            ptr=new int;
        *ptr=val;
    }
    ~circle()
    { delete ptr; ptr=NULL; }

    Int get()
    { return *ptr;}
```

```
dataType Greater(dataType obj)
{
    if(*ptr> *(obj.ptr))
        return (*this);

    return obj;
};

void main()
{
    dataType obj;
    Obj.set(5);
    datatype obj2=obj;
    cout<<obj2.get();
    dataType obj4=obj.Greater(obj2);
}
```

CLASS DISCUSSION

- Can we make copy constructor private?
- Why argument to a copy constructor must be passed as a reference?
- Why argument to a copy constructor should be const?

REFERENCES

- <https://www.geeksforgeeks.org/copy-constructor-in-cpp/>